Chapter-6 (Ian Goodfellow)
KL Divergence, Cross-Entropy, Logistic Regression and SGD

Master's-level Deep Learning

Refai Arafat Rossi

October 2026

Your lesson can begin with two practical problems. The first explains how a neural network measures the mismatch between predicted and target probability distributions. The second explains how a model learns to improve its predictions by updating its parameters.

These examples connect Goodfellow's Sections 3.13, 5.7.1, 5.9, and 6.2.1 without moving into the other topics you plan to teach later. The equations and explanations below follow the book's information-theoretic and maximum-likelihood framework.

Problem 1 — How can a deep learning model learn the correct probability distribution?

1. The problem

Suppose we are developing a deep learning model to classify medical images into two categories: **Class A** and **Class B**.

For a simplified teaching example, assume that the target distribution represents uncertainty in the label:

Class	**Target probability** P	**Model prediction** Q
Class A	0.70	0.60
Class B	0.30	0.40
Total	**1.00**	**1.00**

The model predicts 60% for Class A and 40% for Class B, while the target distribution is 70% and 30%.

Our objective is to measure how different the model's prediction is from the target distribution and give the learning algorithm a loss to minimize.

Question for students: How can we measure this mismatch mathematically?

Goodfellow's Section 3.13 introduces two closely related measures:

- **Cross-entropy:** measures the average information cost incurred when using the model distribution Q to represent outcomes drawn from the target distribution P.
- **Kullback–Leibler (KL) divergence:** measures the extra information cost caused by the mismatch between P and Q.

Both use logarithms of probabilities. For the numerical calculations below, we use the natural logarithm, ln, so the results are expressed in nats.

2. Understand the three related terms

A. Entropy — uncertainty in the target distribution

Entropy is defined as:

$$H(P) = -\sum_i P(i) \log P(i)$$

It measures uncertainty in the target distribution itself. For a binary distribution $P = (0.7, 0.3)$, entropy is:

$$\begin{aligned} H(P) &= -[0.7 \ln(0.7) + 0.3 \ln(0.3)] \\ &= 0.6109 \text{ nats} \end{aligned}$$

B. Cross-entropy — mismatch measured using the model's probabilities

$$\boxed{H(P, Q) = -\sum_i P(i) \log Q(i)}$$

Here, the target probability $P(i)$ weights the logarithm of the probability assigned by the model, $Q(i)$.

C. KL divergence — extra information cost due to the mismatch

$$\boxed{D_{\mathrm{KL}}(P \| Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}}$$

KL divergence is non-negative and equals zero when the two distributions match. It is not a symmetric distance: exchanging P and Q generally changes its value.

3. The essential relationship

This is the most important equation for the first example:

$$\boxed{H(P, Q) = H(P) + D_{\mathrm{KL}}(P \| Q)}$$

In words: Cross-entropy = target entropy + KL divergence.

The target entropy $H(P)$ does not depend on the model parameters. Therefore, when training a model to match a fixed target distribution, minimizing cross-entropy is equivalent to minimizing KL divergence. This is the central relationship in Goodfellow's Section 3.13.

4. Solve the problem step by step

We have:

$$P = (0.7, 0.3), \qquad Q = (0.6, 0.4)$$

Step 1: Calculate cross-entropy.

$$\begin{aligned} H(P, Q) &= -[0.7 \ln(0.6) + 0.3 \ln(0.4)] \\ &= 0.3576 + 0.2749 \\ &= \boxed{0.6325 \text{ nats}} \end{aligned}$$

Step 2: Calculate KL divergence.

$$\begin{aligned} D_{\mathrm{KL}}(P\|Q) &= 0.7 \ln \frac{0.7}{0.6} + 0.3 \ln \frac{0.3}{0.4} \\ &= 0.1079 - 0.0863 \\ &= \boxed{0.0216 \text{ nats}} \end{aligned}$$

Step 3: Verify the relationship.

$$\begin{aligned} H(P) + D_{\mathrm{KL}}(P\|Q) &= 0.6109 + 0.0216 \\ &= \boxed{0.6325} \end{aligned}$$

The result agrees with the calculated cross-entropy.

Step 4: What if the model improves?
Suppose the model changes its prediction to match the target: $Q = (0.7, 0.3)$. Now, $D_{\mathrm{KL}}(P\|Q) = 0$ and $H(P, Q) = H(P) = 0.6109$ nats.

Model distribution Q	**Cross-entropy**	**KL divergence**
$(0.6, 0.4)$	0.6325	0.0216
$(0.7, 0.3)$	0.6109	0.0000

Table 1: Comparison of model predictions.

The target distribution is fixed at $P = (0.7, 0.3)$. When $Q = P$, the mismatch disappears, but the target entropy remains.

5. How does this apply to ordinary deep learning classification?

In conventional supervised classification, an example usually has a single known class rather than a soft target distribution. Suppose the true class is Class A. Its target distribution is one-hot:

$$P = (1, 0)$$

- If the model assigns Class A a probability of 0.70, then $H(P, Q) = -\ln(0.70) = 0.3567$.
- If the model improves its prediction to 0.95, then $H(P, Q) = -\ln(0.95) = 0.0513$.

The loss decreases because the model assigns a higher probability to the correct class.

For a one-hot target, the target entropy is zero. Consequently, cross-entropy equals KL divergence in this particular case:

$$H(P) = 0 \quad \Longrightarrow \quad H(P, Q) = D_{\mathrm{KL}}(P\|Q)$$

This distinction is useful when explaining why KL divergence and cross-entropy are related but are not always numerically identical.

6. Visualize why cross-entropy penalizes wrong predictions

In binary classification, let p be the probability assigned to Class 1. The loss depends on the actual label y.

- For a correct label $y = 1$: $\quad L(p) = -\ln p$
- For a correct label $y = 0$: $\quad L(p) = -\ln(1 - p)$

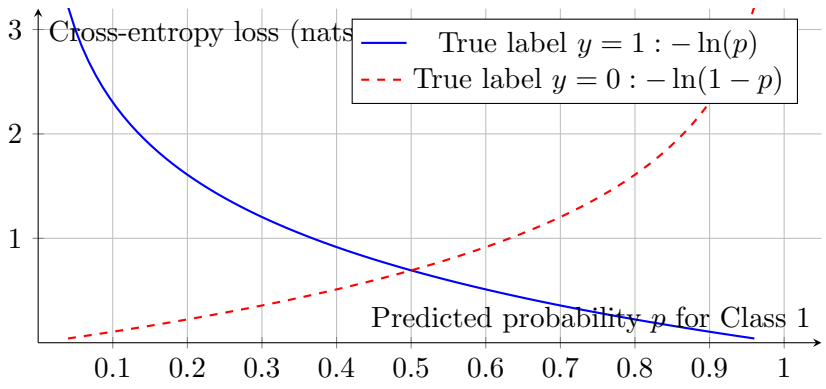


Figure 1: Cross-entropy loss curves as a function of predicted probability p.

For a true label of 1, assigning a probability near 1 produces a small loss. Assigning a probability near 0 produces a large loss. The reverse applies when the true label is 0.

This makes cross-entropy useful for training probabilistic classifiers: it rewards confident correct predictions and penalizes confident incorrect predictions.

Problem 2 — How can logistic regression learn using SGD?

Now we move from measuring prediction error to changing model parameters to reduce that error. This connects Goodfellow's Section 5.7.1 (Logistic Regression), Section 5.9 (Stochastic Gradient Descent), and Section 6.2.1 (Cost Functions).

1. The problem

Suppose we want to classify a network traffic sample as either:

- $y = 0$: benign traffic
- $y = 1$: malicious traffic

For this teaching example, we use just one numerical feature, x, representing a simplified suspicious-traffic score. The model receives $x = 2$, and the known label is $y = 1$, meaning malicious traffic.

Initially, the model has parameters:

$$w = 0, \qquad b = 0$$

Our objective is to use logistic regression and one SGD update to improve the prediction.

2. What is logistic regression?

Although its name contains regression, logistic regression is commonly used for binary classification. It estimates the probability that an input belongs to Class 1.

First, it calculates a linear score:

$$\boxed{z = wx + b}$$

- x is the input feature.

- w is the weight, controlling the effect of the feature.
- b is the bias, shifting the score.
- z is the linear score before the output activation.

The model then applies the sigmoid function:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Therefore,

$$\boxed{p = P(y = 1 \mid x) = \sigma(wx + b)}$$

The sigmoid converts any real-valued score into a probability strictly between 0 and 1. A common classification rule is:

$$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$$

The threshold determines the predicted class, but training normally uses the probability p, not just the final binary prediction.

3. Initial prediction

Given $x = 2$, $y = 1$, $w = 0$, $b = 0$:
First calculate the linear score:

$$z = wx + b = (0)(2) + 0 = 0$$

Apply sigmoid:

$$p = \frac{1}{1 + e^{0}} = \frac{1}{2} = 0.5$$

The model assigns a 50% probability to malicious traffic. This is an uncertain prediction. We now need a loss function to measure the error.

4. Use cross-entropy as the cost function

For binary classification, Goodfellow's maximum-likelihood framework gives the binary cross-entropy loss:

$$\boxed{L(y, p) = -[y \ln p + (1 - y) \ln(1 - p)]}$$

This is the negative log-likelihood of a Bernoulli output distribution, expressed as a loss to minimize. It is the binary form of the cross-entropy objective described in Section 6.2.1.

Because the true label is $y = 1$:

$$\begin{aligned} L &= -[1 \ln(0.5) + 0 \ln(0.5)] \\ &= -\ln(0.5) \\ &= \boxed{0.6931 \text{ nats}} \end{aligned}$$

The initial loss is 0.6931 nats. But how can the model reduce this loss? It must change its weight and bias.

5. Calculate the gradient

A gradient tells us how the loss changes when we change the model's parameters. It tells us which direction to move the parameters to reduce the loss locally.

For sigmoid output and binary cross-entropy, the derivative with respect to the linear score simplifies to:

$$\boxed{\frac{\partial L}{\partial z} = p - y}$$

Since $z = wx + b$, the chain rule gives:

$$\boxed{\frac{\partial L}{\partial w} = (p - y)x} \qquad \text{and} \qquad \boxed{\frac{\partial L}{\partial b} = p - y}$$

These gradients show the direction in which the weight and bias influence the loss.

Calculate the gradients using our example ($p = 0.5$, $y = 1$, and $x = 2$):

- **For the weight:**
$$\frac{\partial L}{\partial w} = (p - y)x = (0.5 - 1)(2) = \boxed{-1}$$
- **For the bias:**
$$\frac{\partial L}{\partial b} = p - y = 0.5 - 1 = \boxed{-0.5}$$

Both gradients are negative. To reduce the loss using gradient descent, we will increase both parameters.

6. Apply stochastic gradient descent (SGD)

Goodfellow's Section 5.9 describes SGD as updating the model parameters in the direction opposite to an estimated gradient:

$$\boxed{\theta \leftarrow \theta - \eta \hat{g}}$$

Here, θ represents all trainable parameters, $\hat{g}$ is the gradient estimate, and η is the learning rate.

Choose the learning rate $\eta = 0.1$:

- **Update the weight:**
$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w} = 0 - (0.1)(-1) = \boxed{0.1}$$
- **Update the bias:**
$$b_{\text{new}} = b - \eta \frac{\partial L}{\partial b} = 0 - (0.1)(-0.5) = \boxed{0.05}$$

The model parameters have changed from $w = 0, b = 0$ to $w = 0.1, b = 0.05$. Because the gradients are negative, subtraction increases the parameters.

7. Calculate the new prediction and loss

Using the updated parameters:

$$z_{\text{new}} = w_{\text{new}}x + b_{\text{new}} = (0.1)(2) + 0.05 = 0.25$$

Apply sigmoid:

$$p_{\text{new}} = \frac{1}{1 + e^{-0.25}} \approx \boxed{0.5622}$$

The predicted probability of malicious traffic has increased from 0.50 to approximately 0.5622.

Recalculate the loss:

$$L_{\text{new}} = -\ln(0.5622) \approx \boxed{0.5759 \text{ nats}}$$

	Loss (nats)	**Predicted attack probability**
Before the update	0.6931	50.00%
After the update	0.5759	56.22%

Table 2: Result of one SGD update.

The loss has decreased by approximately 0.1172 nats after this update. The model has moved toward predicting the correct class with a higher probability.

8. What is the relationship between logistic regression, cross-entropy and SGD?

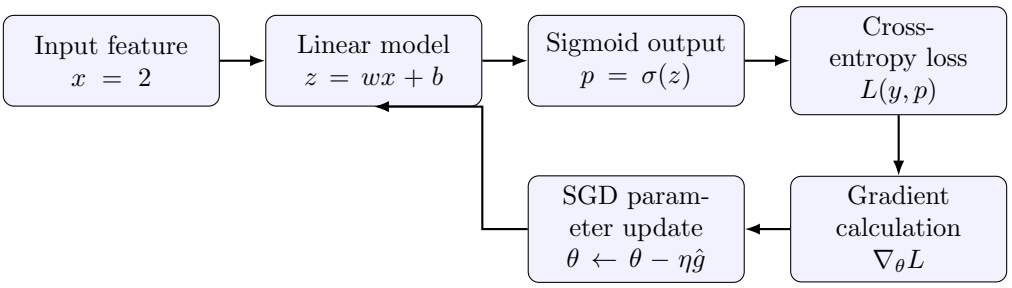


Figure 2: Computational pipeline for training logistic regression.

The roles are distinct:

- **Logistic regression** defines the prediction model: a linear score followed by sigmoid.
- **Cross-entropy** measures how well the predicted probability agrees with the true label.
- **Gradient calculation** determines how the trainable parameters affect the loss.
- **SGD** uses the gradient estimate to update the parameters.

Key message to give students: Cross-entropy tells the learning algorithm how wrong its probability predictions are. The gradient tells it how its parameters influence that error.

SGD then changes those parameters in an attempt to reduce the error.

Primary Textbook Reference: Goodfellow, Bengio and Courville, *Deep Learning*, MIT Press, 2016. The most relevant follow-up reading for these two examples is Section 6.2.1 (*Cost Functions*), which connects cross-entropy to maximum likelihood and model training.

Deep Feedforward Networks

Lecture Notes Based on Chapter 6 of *Deep Learning*

Based on Ian Goodfellow, Yoshua Bengio, and Aaron Courville

Contents

1 Lecture Overview **2**

2 Why Do We Need Deep Learning? **2**

3 What Is a Feedforward Network? **3**

4 The Basic Computational Unit: A Neuron **3**
4.1 Understanding each term 4

5 The Critical Problem: XOR **4**

6 Can One Perceptron Learn XOR? **4**
6.1 Visual understanding of XOR 5
6.2 What does "not linearly separable" mean? 5
6.3 Why is XOR important? 6

7 Solution: Add a Hidden Layer **6**
7.1 Express XOR using AND, OR, and NOT 6
7.2 A numerical threshold-neuron construction 6
7.3 Representation learning 7

8 Why Nonlinear Activation Functions Are Necessary **7**

9 Common Activation Functions **7**

10 ReLU Activation Function **8**
10.1 ReLU examples 8
10.2 ReLU graph 8
10.3 Why is ReLU popular? 9
10.4 A useful note about the ReLU derivative 9

11 Quick Review and Classroom Questions **9**

12 Key Takeaways **9**

1 Lecture Overview

For a Master's class, this chapter is best taught as a gradual transition:

Problem → Perceptron → XOR limitation → Hidden layer → Nonlinearity → ReLU → Deep feedforward network → Forward propagation → Loss → Gradient descent and backpropagation

Target audience: Master's students.
Assumed background: Basic algebra and elementary machine-learning concepts; advanced mathematics is not required.

Learning objectives

By the end of the lecture, students should be able to:

- Explain what a feedforward neural network is.
- Understand why a single perceptron cannot learn XOR.
- Explain how a hidden layer can solve XOR.
- Explain why nonlinear activation functions are necessary.
- Calculate a neuron's output step by step.
- Define and calculate the ReLU activation.
- Describe the architecture of a deep feedforward network.
- Explain forward propagation using equations.
- Describe the roles of weights, biases, and activation functions.
- Solve simple numerical problems involving a neural network.

2 Why Do We Need Deep Learning?

Start with this question:

> Can a machine learn every pattern using a simple linear model?

Consider a student-pass prediction problem. Let

$$x_1 = \text{study hours}, \qquad x_2 = \text{attendance}.$$

The target is

$$y = \begin{cases} 1 & \text{if the student passes,} \\ 0 & \text{if the student fails.} \end{cases}$$

A simple linear model computes

$$z = w_1 x_1 + w_2 x_2 + b,$$

and then applies an activation function:

$$y = \text{activation}(z).$$

For example,

$$z = 2x_1 + 3x_2 - 5.$$

The model learns parameters that determine a decision boundary. A linear model can solve many tasks, but it cannot represent every possible input-output relationship.

3 What Is a Feedforward Network?

A feedforward network processes information from input to output. In a standard feedforward network, information does not travel around recurrent loops.

$$\boxed{\text{Input} \longrightarrow \text{Computation} \longrightarrow \text{Output}}$$

Figure 1 shows a simple network with an input layer, a hidden layer, and an output layer.

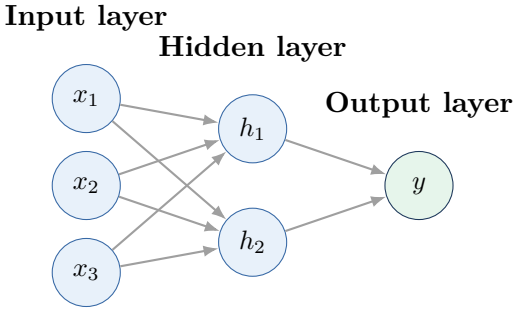


Figure 1: A simple feedforward neural network.

The network transforms

$$x \longrightarrow h \longrightarrow y,$$

where x is the input, h is a hidden representation, and y is the output.

4 The Basic Computational Unit: A Neuron

Before discussing deep networks, students should understand one neuron.

Suppose the inputs are

$$x_1 = 2, \qquad x_2 = 3,$$

the weights are

$$w_1 = 0.5, \qquad w_2 = 0.2,$$

and the bias is $b = 1$.

The neuron first calculates the weighted sum:

$$z = w_1 x_1 + w_2 x_2 + b.$$

Substituting the values,

$$\begin{aligned} z &= (0.5)(2) + (0.2)(3) + 1 \\ &= 1 + 0.6 + 1 \\ &= 2.6. \end{aligned}$$

Thus,

$$\boxed{z = 2.6}.$$

The neuron then applies an activation function:

$$a = f(z).$$

Its overall computation is

$$\boxed{a = f(w^\top x + b)}.$$

This is one of the central equations in the chapter.

4.1 Understanding each term

- x – **input/features:**
$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$
- w – **weights:**
$$w = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix}.$$
- $w^\top x$ – **weighted sum:**
$$w^\top x = (0.5)(2) + (0.2)(3) = 1.6.$$
- b – **bias:** a trainable offset that shifts the neuron's pre-activation value.
- f – **activation function:** a function applied to the weighted sum. A nonlinear activation allows the network to represent nonlinear relationships.

5 The Critical Problem: XOR

XOR means *exclusive OR*. Its output is 1 when the two binary inputs differ and 0 when they are the same.

Table 1: XOR truth table

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Equivalently,
$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

6 Can One Perceptron Learn XOR?

A single perceptron computes
$$z = w_1 x_1 + w_2 x_2 + b$$
and may use a threshold activation:
$$y = \begin{cases} 1 & z > 0, \\ 0 & z \le 0. \end{cases}$$

Its decision boundary is
$$w_1 x_1 + w_2 x_2 + b = 0.$$
For two input features, this boundary is a straight line.

6.1 Visual understanding of XOR

The four XOR input points are

$$(0,0) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1, \quad (1,1) \mapsto 0.$$

The two positive points occupy opposite corners, as do the two negative points. No single straight line can separate both classes.

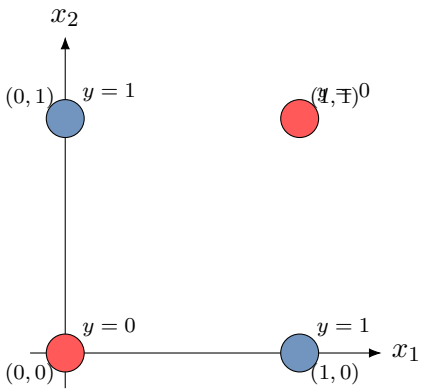


Figure 2: XOR data points. Positive and negative examples alternate across the corners.

Ask the class: Can you draw one straight line that separates both 1s from both 0s?

No

6.2 What does "not linearly separable" mean?

A dataset is linearly separable if a line, plane, or hyperplane can separate its classes.

AND

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

AND is linearly separable.

OR

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

OR is also linearly separable.

XOR is not linearly separable. Therefore,

A single perceptron cannot learn XOR.

6.3 Why is XOR important?

XOR is a small problem that exposes a fundamental limitation: a single linear neuron cannot represent every function. This motivates the use of multiple neurons, hidden layers, and nonlinear activation functions.

7 Solution: Add a Hidden Layer

Instead of connecting inputs directly to the output, add a hidden layer:

$$\text{Input} \longrightarrow \text{Hidden layer} \longrightarrow \text{Output}.$$

The hidden neurons learn intermediate representations. This is the key conceptual transition:

$$\boxed{x \longrightarrow h \longrightarrow y}.$$

7.1 Express XOR using AND, OR, and NOT

A useful logical identity is

$$\boxed{\text{XOR}(x_1, x_2) = (x_1 \,\text{OR}\, x_2) \,\text{AND}\, \text{NOT}(x_1 \,\text{AND}\, x_2)}.$$

Define hidden neurons as

$$h_1 = x_1 \,\text{OR}\, x_2, \qquad h_2 = x_1 \,\text{AND}\, x_2.$$

Then the output is

$$y = h_1 \,\text{AND}\, \text{NOT}(h_2).$$

The hidden layer transforms the original input into a representation that makes the final decision possible.

Table 2: XOR represented by hidden features

x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{AND}$	$y = h_1 \wedge \neg h_2$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

7.2 A numerical threshold-neuron construction

The logic above can be implemented with threshold neurons. Let

$$\text{step}(z) = \begin{cases} 1 & z > 0, \\ 0 & z \leq 0. \end{cases}$$

The OR hidden neuron can be defined as

$$h_1 = \text{step}(x_1 + x_2 - 0.5),$$

and the AND hidden neuron as

$$h_2 = \text{step}(x_1 + x_2 - 1.5).$$

The output neuron is

$$y = \text{step}(h_1 - h_2 - 0.5).$$

Check all four cases:

- $(0,0)$: $h_1 = 0$, $h_2 = 0$, so $y = 0$.
- $(0,1)$: $h_1 = 1$, $h_2 = 0$, so $y = 1$.
- $(1,0)$: $h_1 = 1$, $h_2 = 0$, so $y = 1$.
- $(1,1)$: $h_1 = 1$, $h_2 = 1$, so $y = 0$.

Thus, the hidden layer enables a network of threshold neurons to represent XOR.

7.3 Representation learning

The original input representation is (x_1, x_2). The hidden representation is (h_1, h_2). The network learns useful intermediate features rather than relying only on the original input coordinates.

8 Why Nonlinear Activation Functions Are Necessary

Suppose every layer performs only an affine transformation:

$$h = W_1 x + b_1, \qquad y = W_2 h + b_2.$$

Substituting the first equation into the second gives

$$\begin{aligned} y &= W_2(W_1 x + b_1) + b_2 \\ &= W_2 W_1 x + W_2 b_1 + b_2. \end{aligned}$$

This is still an affine transformation of x. Therefore, stacking affine layers without nonlinear activations does not provide the expressive power expected from a deep network.

With a nonlinear activation function,

$$\boxed{h = f(Wx + b)},$$

the network can learn nonlinear relationships. In short,

$$\text{many affine layers without nonlinearities} \;\equiv\; \text{one affine transformation.}$$

9 Common Activation Functions

Common activation functions include:

- Step function
- Sigmoid
- Hyperbolic tangent (tanh)
- Rectified Linear Unit (ReLU)
- Leaky ReLU
- Softmax

For this lecture, the main focus is ReLU.

10 ReLU Activation Function

ReLU stands for *Rectified Linear Unit*. It is defined as

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

or, equivalently,

$$\text{ReLU}(x) = \begin{cases} 0 & x < 0, \\ x & x \geq 0. \end{cases}$$

In simple terms:

- Negative input: output 0.
- Positive input: keep the input unchanged.
- Zero input: output 0.

10.1 ReLU examples

$$\text{ReLU}(5) = 5$$

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(0) = 0$$

$$\text{ReLU}(2.7) = 2.7$$

$$\text{ReLU}(-8.4) = 0$$

Question for students: What is $\text{ReLU}(-2.5)$?

$$\boxed{\text{ReLU}(-2.5) = 0}$$

10.2 ReLU graph

The function is flat at zero for negative inputs and follows the line $y = x$ for nonnegative inputs.

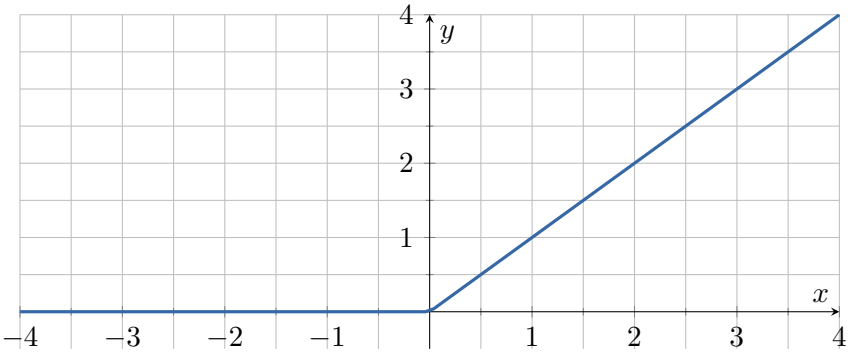


Figure 3: ReLU: $\text{ReLU}(x) = \max(0, x)$.

10.3 Why is ReLU popular?

1. **Simple:** $\mathrm{ReLU}(x) = \max(0, x)$.
2. **Computationally inexpensive:** it requires a comparison and selection rather than an exponential calculation.
3. **Nonlinear:** despite being piecewise linear, it allows a network to represent nonlinear functions when combined across layers.
4. **Often helps optimization:** on its positive side, its derivative is 1, so it avoids the positive-side saturation associated with sigmoid and tanh.

10.4 A useful note about the ReLU derivative

For $x < 0$, the derivative is 0. For $x > 0$, the derivative is 1. At $x = 0$, ReLU is not differentiable in the classical sense; implementations choose a subgradient convention, commonly 0.

$$\mathrm{ReLU}'(x) = \begin{cases} 0 & x < 0, \\ 1 & x > 0. \end{cases}$$

A limitation is that a neuron that remains in the negative region for all its inputs may receive zero gradient through ReLU and stop updating through that path. This is often called the *dying ReLU* problem.

11 Quick Review and Classroom Questions

1. What is a feedforward neural network?
2. What do weights and biases do in a neuron?
3. Why can a single perceptron represent AND and OR but not XOR?
4. What does it mean for a dataset to be linearly separable?
5. How does a hidden layer help solve XOR?
6. Why are nonlinear activation functions necessary?
7. Calculate $\mathrm{ReLU}(-4)$, $\mathrm{ReLU}(0)$, and $\mathrm{ReLU}(3.5)$.
8. What can happen if a network stacks many affine layers without nonlinear activation functions?

12 Key Takeaways

- A neuron computes a weighted sum plus a bias and applies an activation function.
- A single perceptron has a linear decision boundary and cannot represent XOR.
- A hidden layer can construct intermediate features that make XOR solvable.
- Stacking affine layers alone does not create a nonlinear model.
- Nonlinear activations are essential for the expressive power of deep feedforward networks.
- ReLU is simple and computationally efficient, and often supports effective gradient-based training.

Reference

Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning.* MIT Press, 2016. Chapter 6, "Deep Feedforward Networks." Available at: https://www.deeplearningbook.org/contents/mlp.html

Deep Learning Lecture: Gradient-Based Learning, Output Units and ReLU

Refai Arafat Rossi

October 2026

We will use one simple cybersecurity classification problem throughout the lesson: building a neural network that classifies network traffic as benign or malicious, and then extends the same task to distinguish benign traffic, DDoS attacks and port scans.
This single example will help your students understand why a neural network needs a cost function, why it uses different output units for different tasks, how hidden units extract features, and how ReLU supports gradient-based learning. These explanations follow the organization and core equations of Goodfellow's Chapter 6.

1 Start with the problem: How can a neural network recognize an attack?

Suppose a network administrator wants an AI model to detect suspicious traffic. We provide two numerical features for every network-flow sample.

Feature	**Meaning**	**Example value**
x_1	Normalized packet-rate feature	2
x_2	Normalized connection-failure feature	1

The model receives the input vector

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

For our first task, the target is:

$$y = \begin{cases} 0 & \text{Benign traffic} \\ 1 & \text{Malicious traffic} \end{cases}$$

Assume the known training label is $y = 1$. The model must learn from examples to predict the probability $P(y = 1 \mid \mathbf{x})$ rather than merely produce an arbitrary numerical score.
Our four main questions are:

1. How does the model produce a probability?
2. How do we measure the difference between its prediction and the correct answer?
3. How does the model change its weights to reduce that error?
4. Why does it need hidden units and activation functions such as ReLU?

1.1 A simple neural network diagram

- **Input layer:** $\mathbf{x} = [2, 1]^T$ (Packet-rate and connection-failure features)
- **Hidden layer (ReLU):** $\mathbf{h} = \text{ReLU}(W^T\mathbf{x} + \mathbf{b})$ (Transforms the input into learned features)
- **Output layer:** Sigmoid for binary classification, or Softmax for multiclass classification
- **Cost function:** $J(\theta) = -\log P(y \mid \mathbf{x}; \theta)$ (Measures the loss for the known training label)

The neural network makes a prediction in the forward pass. It then calculates the cost, computes gradients and adjusts its parameters during training. The essential idea in Section 6.2 is that a neural network is trained by defining a model, choosing a cost function and using an optimization procedure to reduce that cost.

2 Section 6.2 — Gradient-based learning

2.1 What does gradient-based learning mean?

A neural network contains trainable parameters, including weights and biases. Collect them into a parameter vector θ. The network's prediction depends on these parameters:

$$\hat{y} = f(\mathbf{x}; \theta)$$

The cost function tells us how poorly the prediction matches the target:

$$J(\theta) = L\big(f(\mathbf{x}; \theta), y\big)$$

The training objective is to find parameters that minimize the cost:

$$\boxed{\theta^* = \arg\min_{\theta} J(\theta)}$$

A gradient tells us how the cost changes as the parameters change:

$$\nabla_\theta J(\theta)$$

The basic gradient-descent update is

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta J}$$

where η is the learning rate.
Think of the cost as the height of a landscape. The gradient indicates the direction of the steepest increase, so subtracting the gradient moves the parameters locally in the direction of decreasing cost.

2.2 What happens during training?

1. **Step 1: Forward pass.** Process the input through the layers and calculate the prediction.
2. **Step 2: Calculate the cost.** Compare the predicted probability with the known label using a suitable loss function.
3. **Step 3: Calculate gradients.** Determine how the cost changes with the weights and biases.

4. **Step 4: Update the parameters.** Use gradient descent or an optimizer such as SGD to adjust the parameters, then repeat.

These steps repeat across many training examples. The detailed algorithm for efficiently calculating gradients throughout a network is backpropagation, covered separately in Goodfellow's Section 6.5.

2.3 The cost function in Goodfellow's framework

Goodfellow expresses the maximum-likelihood training objective as

$$\boxed{J(\theta) = -\mathbb{E}_{(\mathbf{x},y)\sim\hat{p}_{\text{data}}} \left[\log p_{\text{model}}(y \mid \mathbf{x};\theta)\right]}$$

Here:

- $\hat{p}_{\text{data}}$ represents the empirical distribution of the training data.
- $p_{\text{model}}(y \mid \mathbf{x};\theta)$ is the probability assigned by the model to the correct target.
- The negative logarithm penalizes the model when it assigns a low probability to the correct answer.

For a dataset containing N examples, the empirical cost is commonly written as

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\text{model}}(y_i \mid \mathbf{x}_i;\theta)$$

This is the negative log-likelihood, equivalent to the appropriate cross-entropy objective for the modeled output distribution.

3 Section 6.2.1 — Cost function: Cross-entropy and logistic regression

Now consider the binary traffic-classification problem again. The model estimates the probability that the traffic is malicious. The true class is $y = 1$. Suppose the model assigns a probability of 0.80 to the malicious class. How do we measure the quality of this prediction?

3.1 Cross-entropy loss for binary classification

For a binary target $y \in \{0, 1\}$, binary cross-entropy is

$$\boxed{L(y,p) = -\left[y \log p + (1-y)\log(1-p)\right]}$$

where $p = P(y = 1 \mid \mathbf{x})$. When the true label is $y = 1$, this expression simplifies to $L = -\log p$. For our example, $p = 0.80$:

$$L = -\ln(0.80) = \boxed{0.2231}$$

Now suppose the model is confidently wrong and predicts $p = 0.10$:

$$L = -\ln(0.10) = \boxed{2.3026}$$

The second model receives a much larger loss because it assigns a low probability to the correct class.

Teaching point: Cross-entropy does not simply ask whether the predicted class is right or wrong. It also considers the probability assigned to the correct answer. For easier classroom interpretation, these examples use natural logarithms and express loss in nats.

Predicted probability of malicious traffic	True label	Cross-entropy loss
0.95	1	0.0513
0.80	1	0.2231
0.50	1	0.6931
0.10	1	2.3026

3.2 What is logistic regression?

Logistic regression is a binary classification model. It first calculates a linear score and then converts it into a probability using the sigmoid function.
The linear score is

$$\boxed{z = \mathbf{w}^T\mathbf{x} + b}$$

The predicted probability is

$$\boxed{p = P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}}$$

The model can then classify the input according to a chosen decision threshold, often 0.5. Consider an example with $z = 1$:

$$p = \frac{1}{1 + e^{-1}} \approx 0.7311$$

The model predicts a 73.11% probability of malicious traffic. With true label $y = 1$, the loss becomes

$$L = -\ln(0.7311) \approx 0.3133$$

The combination of a sigmoid output and cross-entropy loss is a natural choice because it defines a probability distribution for the binary target and trains the model through maximum likelihood.

3.3 How is logistic regression related to a deep neural network?

In standard logistic regression, the linear score uses the original input features:

$$p = \sigma(\mathbf{w}^T\mathbf{x} + b)$$

In a neural network, the hidden layers first transform the input into a learned representation $\mathbf{h}$. The output layer then applies logistic regression to those learned features:

$$\boxed{p = \sigma(\mathbf{w}^T\mathbf{h} + b)}$$

The important difference is that the neural network learns a representation $\mathbf{h}$, while standard logistic regression directly uses $\mathbf{x}$. A neural network can therefore combine nonlinear hidden-layer transformations with a familiar logistic-regression output layer.

4 Section 6.2.2 — Output units

Goodfellow emphasizes that the choice of output units is closely connected to the cost function. The output layer must represent the type of target variable the model is learning to predict. For our network-security example, we will consider two different tasks:

- **Task A:** Is the traffic benign or malicious?
- **Task B:** Is the traffic benign, a DDoS attack or a port scan?

These tasks need different output distributions.

5 Section 6.2.2.2 — Sigmoid units for Bernoulli output distributions

5.1 When should we use sigmoid?

Use a sigmoid output when predicting a binary random variable, such as benign versus malicious, spam versus not spam, or disease detected versus not detected. A Bernoulli distribution is completely specified by one probability:

$$p = P(y = 1 \mid \mathbf{x})$$

The probability of the other outcome is $P(y = 0 \mid \mathbf{x}) = 1 - p$.
The sigmoid output unit is

$$\boxed{\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}}$$

where $z = \mathbf{w}^T\mathbf{h} + b$. The resulting Bernoulli distribution is

$$\boxed{P(y \mid \mathbf{x}) = \hat{y}^{\,y}(1 - \hat{y})^{1-y}}$$

Its negative log-likelihood gives the binary cross-entropy loss.

5.2 Understand the sigmoid graph

The sigmoid has an S-shaped curve. Its output lies strictly between 0 and 1 for finite inputs.

Logit z	**Sigmoid output**	**Interpretation**
-2	0.1192	Low probability of an attack
0	0.5000	Equal predicted probabilities
1	0.7311	Higher probability of an attack
2	0.8808	High probability of an attack

The sigmoid output is a probability estimate, not a guarantee that the prediction is correct. A threshold is needed to convert the probability into a class label. A threshold of 0.5 is common, but operational security systems may use a different threshold depending on the relative costs of false positives and false negatives.

5.3 Calculate one complete prediction

Return to the teaching example. Suppose the hidden layer produces features

$$\mathbf{h} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

For an illustrative set of output weights, let $z = 0.7h_1 + 0.4h_2 - 0.8$. Substituting the values:

$$z = 0.7(2) + 0.4(1) - 0.8 = 1$$

Therefore, $p = \sigma(1) = 0.7311$. For a malicious sample with $y = 1$:

$$L = -\ln(0.7311) = 0.3133$$

The model is leaning toward the correct answer, but the loss indicates that it has not assigned probability 1 to the target.

5.4 How does this prediction improve through gradient-based learning?

For sigmoid output combined with binary cross-entropy, the derivative of the loss with respect to the output logit simplifies to

$$\boxed{\frac{\partial L}{\partial z} = p - y}$$

In our example,

$$\frac{\partial L}{\partial z} = 0.7311 - 1 = -0.2689$$

The negative gradient indicates that increasing z locally decreases the loss.
To illustrate one update, let the learning rate be 0.1 and update only the output weights and bias. Their gradients are

$$\frac{\partial L}{\partial w_j} = (p - y)h_j, \qquad \frac{\partial L}{\partial b} = p - y$$

Parameter	Before update	After one update
w_1	0.7000	0.7538
w_2	0.4000	0.4269
b	-0.8000	-0.7731

The new logit is approximately 1.1614, producing $p_{\text{new}} \approx 0.7615$ and $L_{\text{new}} \approx -\ln(0.7615) = 0.2725$. The loss has decreased from approximately 0.3133 to 0.2725. This is one illustrative update; a complete training procedure also propagates gradients through the hidden layer.

6 Section 6.2.2.3 — Softmax units for multinoulli output distributions

Now change the classification task. Instead of two categories, we want the model to distinguish: 1. Benign traffic 2. DDoS attack 3. Port scan
These are mutually exclusive classes in this example: each input has one correct class. A single sigmoid output is no longer the natural way to represent the complete distribution over these three classes. Instead, we use a softmax output layer.

6.1 The softmax equation

Suppose the network produces K logits: $\mathbf{z} = [z_1, z_2, \ldots, z_K]$. Softmax converts these scores into probabilities:

$$\boxed{p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$$

The softmax probabilities satisfy two conditions: $0 < p_k < 1$ and

$$\boxed{\sum_{k=1}^{K} p_k = 1}$$

The scores before softmax are called logits. They are not probabilities; they can be any real numbers.

6.2 Work through a three-class example

Suppose the network produces these logits for a single traffic sample: Benign: 0, DDoS: 2, Port scan: 1. The exponentials are approximately $e^0 = 1$, $e^2 = 7.389$, $e^1 = 2.718$. Their sum is approximately 11.107. The probabilities are therefore:

$$P(\text{Benign} \mid \mathbf{x}) = \frac{1}{11.107} \approx 0.090$$

$$P(\text{DDoS} \mid \mathbf{x}) = \frac{7.389}{11.107} \approx 0.665$$

$$P(\text{Port scan} \mid \mathbf{x}) = \frac{2.718}{11.107} \approx 0.245$$

The largest probability is assigned to DDoS. The predicted class is therefore DDoS.

6.3 Softmax and cross-entropy

Suppose the true class is DDoS. Its one-hot target vector is $\mathbf{y} = [0, 1, 0]$. For a single-label, multiclass classification problem, cross-entropy is

$$\boxed{L = -\sum_{k=1}^{K} y_k \log p_k}$$

Since only the DDoS target entry equals 1,

$$L = -\ln(0.665) \approx \boxed{0.408}$$

The loss decreases when the model assigns a higher probability to the correct class.
For softmax combined with cross-entropy, the corresponding logit gradient is also simple:

$$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$$

6.4 Sigmoid versus softmax

Property	**Sigmoid output**	**Softmax output**
Typical application	Binary classification	Single-label multiclass classification
Output structure	One probability for positive class	One probability per class
Probability relationship	Negative class has probability $1 - p$	All class probabilities sum to 1
Distribution	Bernoulli	Multinoulli / categorical
Loss	Binary cross-entropy	Multiclass cross-entropy
Example	Benign versus attack	Benign, DDoS or port scan

Important distinction: If several labels can be true simultaneously, such as one traffic sample having multiple independent attributes, separate sigmoid outputs may be more appropriate than softmax. Softmax is intended for a probability distribution over mutually exclusive alternatives.

7 Section 6.3 — Hidden units

So far we have concentrated on the output layer. But where do the hidden features $\mathbf{h}$ come from? A hidden layer transforms the original input using weights, biases and an activation function. In Goodfellow's notation, a typical layer can be written as

$$\boxed{\mathbf{h} = g(W^T\mathbf{x} + \mathbf{b})}$$

where W contains the layer's weights, $\mathbf{b}$ contains its biases, g is the activation function, and $\mathbf{h}$ contains the resulting hidden activations.

7.1 Calculate a hidden layer using ReLU

Let the input be $\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. For an illustrative hidden layer, define:

$$z_1 = x_1 + x_2 - 1 = 2$$

$$z_2 = x_1 - x_2 = 1$$

These are linear scores. The activation function transforms each score into a hidden activation. We will use the Rectified Linear Unit (ReLU).

8 Section 6.3.1 — ReLU and generalization

8.1 What is ReLU?

The ReLU activation function is defined as

$$\boxed{g(z) = \max(0, z)}$$

It follows a simple rule:

$$g(z) = \begin{cases} 0, & z \leq 0 \\ z, & z > 0 \end{cases}$$

Apply ReLU to our two hidden-unit scores: $h_1 = \text{ReLU}(2) = 2$ and $h_2 = \text{ReLU}(1) = 1$. Therefore, the hidden representation is

$$\boxed{\mathbf{h} = [2, 1]^T}$$

8.2 ReLU and sigmoid comparison

- **ReLU:** Returns zero for negative input, passes positive input unchanged, has derivative 1 when $z > 0$ and 0 when $z < 0$, not differentiable at zero (handled by subgradient convention).
- **Sigmoid:** Produces output between 0 and 1, useful for binary probability, saturates toward 0 and 1 for extreme inputs, can produce very small gradients.

Goodfellow identifies ReLU as an excellent default for hidden units because its active region behaves linearly and its gradient does not diminish merely because the input is very positive.

Linear score z	**ReLU output** $\max(0, z)$	**Interpretation**
-2	0	Unit inactive for this input
-0.5	0	Unit inactive for this input
0	0	Output is zero
0.5	0.5	Positive activation
2	2	Positive activation

8.3 ReLU in a neural network: a numerical example

8.4 Why does ReLU help training?

Consider the derivative of ReLU:

$$g'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

When a ReLU unit is active, its derivative is 1. This allows the gradient to pass through that activation without the saturation problem associated with sigmoid hidden units. ReLU is also piecewise linear.

8.5 What does generalization mean?

Generalization is the ability of a trained model to perform well on previously unseen examples drawn from the relevant data distribution. ReLU does not guarantee generalization; it helps make hidden layers easier to optimize, but generalization also depends on training data, model capacity, architecture, and validation.

9 Put all the concepts together

Concept	**Purpose**
Gradient-based learning	Optimizes the neural network's parameters
Cross-entropy	Measures mismatch between predicted output distribution and target
Logistic regression	Produces binary-class probability from a linear score
Sigmoid output	Represents a Bernoulli probability
Softmax output	Represents a probability distribution across mutually exclusive classes
Hidden units	Learn intermediate feature representations
ReLU	Introduces nonlinearity while allowing gradients to pass
Generalization	Measures performance on unseen data

Primary reading: Goodfellow, Bengio and Courville — Deep Learning, Chapter 6: Deep Feedforward Networks. Focus on Sections 6.2, 6.2.1, 6.2.2.2, 6.2.2.3, 6.3 and 6.3.1.

Deep Learning Lecture: Architecture Design, Universal Approximation and Backpropagation

Refai Arafat Rossi

Deep Learning, Chapter 6, Sections 6.4, 6.4.1, 6.5, 6.5.2 and 6.5.4.

October 2026

We will continue with the same network-security problem: classifying network traffic as Benign, DDoS or Port scan. We will not use the XOR example.
The lesson will build one small, fully connected multilayer perceptron (MLP) and calculate its forward pass, loss, gradients and parameter updates by hand. We will use this same example to explain architecture design, the universal approximation theorem, the Jacobian matrix, the chain rule and backpropagation. Goodfellow presents deep feedforward networks as function-approximation models and explains backpropagation as an efficient way to calculate gradients through the network.

1 Begin with our classification problem

A network administrator wants to classify traffic based on two numerical features.

Input feature	**Meaning**	**Example value**
x_1	Normalized packet-rate feature	2
x_2	Normalized connection-failure feature	1

The input vector is

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

The network must predict one of three mutually exclusive classes:

$$C_1 = \text{Benign}$$
$$C_2 = \text{DDoS}$$
$$C_3 = \text{Port scan}$$

For our training example, the correct class is DDoS. Its one-hot target vector is

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

A one-hot vector assigns 1 to the correct class and 0 to the others. Our goal is to build a network that calculates $P(C_k \mid \mathbf{x}), k = 1, 2, 3$ and learns parameters that give higher probabilities to the correct classes.

2 Section 6.4 — Architecture design

2.1 What is a neural network architecture?

Architecture describes how a neural network is organized: its layers, the number of units in each layer, the connections between them, and the operations used to transform the input into the output. For our illustrative problem, choose this architecture:

- **Input layer:** 2 features (Packet rate and connection failures)
- **Hidden layer:** 2 ReLU units (Learns combinations of the input features)
- **Output layer:** 3 Softmax units (Benign, DDoS and Port scan probabilities)

We call this a 2–2–3 MLP: two inputs, two hidden units and three output units. It has one hidden layer. The hidden layer uses ReLU, while the output layer uses Softmax because the task has three mutually exclusive classes. This is a deliberately small teaching network. A practical traffic classifier would normally use more features, and its architecture would be selected using validation experiments.

2.2 What does a fully connected layer mean?

A fully connected layer means that each unit in the next layer receives input from every unit in the previous layer. In our network:

- Each of the two hidden units receives both input features.
- Each of the three output units receives both hidden activations.

For the first layer,

$$\boxed{\mathbf{z}^{(1)} = \mathbf{x}^T W^{(1)} + \mathbf{b}^{(1)}}$$

and

$$\boxed{\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)})}$$

For the output layer,

$$\boxed{\mathbf{z}^{(2)} = \mathbf{h}^T W^{(2)} + \mathbf{b}^{(2)}}$$

where the transposes and vector arrangement here follow our column-vector notation for $\mathbf{x}, \mathbf{z}^{(1)}, \mathbf{h}$, and $\mathbf{z}^{(2)}$. Equivalently, the forward pass can be written using row vectors as $\mathbf{x}^T W^{(1)}$ and $\mathbf{h}^T W^{(2)}$. Softmax converts the output logits into the three probabilities:

$$\boxed{p_k = \frac{e^{z_k^{(2)}}}{\sum_{j=1}^{3} e^{z_j^{(2)}}}}$$

The output with the largest probability becomes the predicted class.

2.3 How many trainable parameters does this network have?

Architecture design also determines the number of trainable weights and biases.

- **Input-to-hidden layer:** $2 \times 2 = 4$ weights. There are two hidden biases, giving $4 + 2 = 6$ parameters.
- **Hidden-to-output layer:** $2 \times 3 = 6$ weights. There are three output biases, giving $6 + 3 = 9$ parameters.

Thus, the network contains

$$\boxed{6 + 9 = 15 \text{ trainable parameters}}$$

This count matters because architecture influences model capacity, memory usage, computational cost and the amount of training data that may be needed.

2.4 How should we choose an architecture?

There is no single architecture that is best for every task. Goodfellow discusses width, depth and connectivity as design decisions for feedforward networks, with validation performance guiding practical choices.

Design decision	**Teaching interpretation**	**Our example**
Input width	How many features enter the model?	2
Hidden width	How many units transform the features at each hidden layer?	2
Depth	How many successive transformations are used?	1 hidden layer
Output width	How many outputs represent the possible classes?	3
Connectivity	How do units in adjacent layers connect?	Fully connected

If the model is too small, it may fail to capture important patterns. If it is unnecessarily large, it may become more expensive to train and more susceptible to overfitting. The choice should consider the task's complexity, available data and computational resources.

3 Section 6.4.1 — Universal approximation properties and depth

3.1 What is the universal approximation theorem?

The universal approximation theorem tells us that a feedforward neural network with a suitable nonlinear hidden-unit activation and enough hidden units can approximate a very broad class of functions to arbitrary accuracy under appropriate conditions. For example, a sufficiently wide network with one hidden layer can approximate any continuous function on a compact input domain to any specified positive approximation tolerance, given a suitable activation function. ReLU networks also have universal approximation results.
In simple language: A neural network does not need a specially designed formula for every pattern. Its weights, biases and nonlinear activation functions can combine to approximate complicated input-output relationships. For our problem, the target function maps traffic features to a class or class-probability distribution:

$$f^* : \mathbf{x} \longrightarrow \begin{bmatrix} P(\text{Benign} \mid \mathbf{x}) \\ P(\text{DDoS} \mid \mathbf{x}) \\ P(\text{Port scan} \mid \mathbf{x}) \end{bmatrix}$$

The network learns an approximation to this function from labeled examples.

3.2 Why not use only a linear model?

Without nonlinear hidden activations, stacking linear layers still gives a linear transformation overall. For example, if $\mathbf{h} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$ and $\mathbf{z} = W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)}$, then substituting the first equation into the second gives another affine function of $\mathbf{x}$. The extra layer alone does not create a nonlinear feature transformation. ReLU changes this:

$$\mathbf{h} = \text{ReLU}\left(W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

Because ReLU is nonlinear, the resulting network can represent nonlinear relationships between input features and output classes. For example, the distinction between benign traffic and a DDoS attack might depend on combinations of packet rates, connection failures and other traffic characteristics, rather than on any single feature alone.

3.3 Why do we need deep networks if one hidden layer is sufficient in theory?

The theorem concerns the network's representational capability. It does not guarantee that a practical network will learn the desired function efficiently. A single hidden layer may require an enormous number of units for some functions. Additional layers can represent certain compositional or hierarchical functions more efficiently, sometimes using far fewer parameters than a shallow network. Goodfellow notes that deeper ReLU networks can represent functions with a number of linear regions that grows rapidly with depth.

- **Shallow network:** One hidden layer can theoretically approximate complex functions, but may need many hidden units.
- **Deep network:** Multiple hidden layers can build successive representations, potentially capturing complex structures more efficiently.

There are three crucial limitations to the theorem:

1. It does not guarantee that a training algorithm will find the right parameters.
2. It does not guarantee good generalization to unseen traffic.
3. It does not say that deeper networks are always better or easier to optimize.

In practice, depth and width are hyperparameters that must be selected and evaluated using data.

4 Section 6.5 — Backpropagation

We now understand the architecture and how it can represent a classification function. The next question is: How can the network learn the values of all 15 parameters? The answer is backpropagation combined with an optimization method such as stochastic gradient descent (SGD). Goodfellow makes an important distinction: backpropagation calculates gradients; gradient descent or another optimizer uses those gradients to update the parameters. Backpropagation itself is a general procedure for efficiently evaluating derivatives through a computational graph.

4.1 Forward propagation versus backward propagation

- **Forward propagation:** Calculate activations, logits, probabilities and loss by moving from the input layer toward the output layer.
- **Backward propagation:** Start with the derivative of the loss and propagate gradients back through the output and hidden layers to determine how each parameter affects the loss.
- **Parameter update:** Use the calculated gradients to update the weights and biases, then repeat the process.

4.2 Our network's forward pass

For clarity, we will use column vectors throughout the mathematical derivation. The input is

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

We choose the following initial weights and biases. These are illustrative values designed for hand calculation, not the result of training on a real network-traffic dataset.
Input-to-hidden weights:

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 1.0 \end{bmatrix}, \qquad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

The pre-activation values for the hidden layer are $\mathbf{z}^{(1)} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$. Substituting the values:

$$\begin{aligned} \mathbf{z}^{(1)} &= \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}} \end{aligned}$$

Both activations are positive. Applying ReLU therefore gives

$$\boxed{\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}}$$

Hidden-to-output weights:

$$W^{(2)} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 3 & 0 \end{bmatrix}, \qquad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

The three logits are

$$\begin{aligned} \mathbf{z}^{(2)} &= W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)} \\ &= \begin{bmatrix} 0 & 0 \\ 1 & 3 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}} \end{aligned}$$

These correspond to Benign, DDoS and Port scan, respectively. Applying Softmax yields

$$\mathbf{p} = \begin{bmatrix} 0.090031 \\ 0.665241 \\ 0.244728 \end{bmatrix}$$

For the true DDoS label $\mathbf{y} = [0, 1, 0]^T$, the cross-entropy loss is

$$\begin{aligned} L &= -\sum_{k=1}^{3} y_k \ln p_k \\ &= -\ln(0.665241) \\ &= \boxed{0.407606} \end{aligned}$$

This is the starting loss that backpropagation will help us reduce.

5 Section 6.5.2 — Chain rule of calculus, Jacobian matrix and gradient

5.1 The chain rule for a simple calculation

Suppose $u = 2x, \quad L = u^2$. Then $L = (2x)^2$. The chain rule gives

$$\boxed{\frac{dL}{dx} = \frac{dL}{du}\frac{du}{dx}}$$

Since $\frac{dL}{du} = 2u, \quad \frac{du}{dx} = 2$, we obtain $\frac{dL}{dx} = 2u \times 2 = 4u$. For $x = 3, u = 6$, and the derivative is 24. The key idea is that when one variable depends on another through intermediate operations, we multiply the local derivatives along the path. A neural network is a larger composition of operations. Backpropagation applies this same principle systematically to every relevant weight and bias.

5.2 What is a gradient?

For a scalar loss L and parameter vector $\boldsymbol{\theta}$, the gradient is

$$\boxed{\nabla_{\boldsymbol{\theta}} L = \begin{bmatrix} \partial L/\partial\theta_1 \\ \partial L/\partial\theta_2 \\ \vdots \\ \partial L/\partial\theta_n \end{bmatrix}}$$

Each entry measures how the loss changes when one parameter changes, while the other parameters are held fixed. In our network, the parameters include all entries of $W^{(1)}, W^{(2)}$ and both bias vectors. The gradient tells us how each of these parameters influences the loss on our training example.

5.3 What is a Jacobian matrix?

A gradient describes the derivatives of a scalar function with respect to its inputs. A Jacobian generalizes the idea to a function that maps a vector to another vector. Suppose $\mathbf{y} = f(\mathbf{x})$ where $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$. The Jacobian is

$$\boxed{J_f(\mathbf{x}) = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \cdots & \frac{\partial y_n}{\partial x_m} \end{bmatrix}}$$

It records how every output component responds to every input component. For a scalar loss L that depends on $\mathbf{y}$, the vector form of the chain rule is

$$\boxed{\nabla_{\mathbf{x}} L = J_f(\mathbf{x})^T \nabla_{\mathbf{y}} L}$$

Here the superscript T denotes the transpose. This is the central mathematical operation behind backpropagation: a gradient is propagated from one operation to its inputs using the local Jacobian.

5.4 The Jacobian of Softmax

Recall that Softmax produces $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Its Jacobian has elements

$$\boxed{\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)}$$

where $\delta_{ij} = 1$ if $i = j$, and 0 otherwise. For our probabilities $\mathbf{p} = [0.090031, 0.665241, 0.244728]^T$, the Softmax Jacobian is approximately

$$J_{\text{softmax}} \approx \begin{bmatrix} 0.081925 & -0.059892 & -0.022033 \\ -0.059892 & 0.222695 & -0.162803 \\ -0.022033 & -0.162803 & 0.184836 \end{bmatrix}$$

Every row describes how one predicted probability changes as the three logits change. This matters because changing one logit can influence more than one class probability: Softmax normalizes all classes together so their probabilities sum to one.

5.5 Combine the Softmax Jacobian with cross-entropy

For our one-hot target, the loss is $L = -\sum_i y_i \ln p_i$. Its derivative with respect to the probabilities is $\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}$. Applying the chain rule through Softmax gives

$$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i} \frac{\partial p_i}{\partial z_j}$$

This simplifies to the important result

$$\boxed{\frac{\partial L}{\partial z_j} = p_j - y_j}$$

In vector form,

$$\boxed{\boldsymbol{\delta}^{(2)} = \mathbf{p} - \mathbf{y}}$$

For our DDoS example:

$$\boldsymbol{\delta}^{(2)} = \begin{bmatrix} 0.090031 \\ 0.665241 \\ 0.244728 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$= \boxed{\begin{bmatrix} 0.090031 \\ -0.334759 \\ 0.244728 \end{bmatrix}}$$

These are the gradients of the loss with respect to the three output logits. They initiate the backward calculation.

6 Section 6.5.4 — Backpropagation in a fully connected MLP

Now we calculate the gradients for every layer in our 2–2–3 network. The important rule is to start at the output, where the loss is known, then move backward toward the input layer.

6.1 Calculate the gradients for the output layer

We already calculated the output-logit gradient:

$$\boldsymbol{\delta}^{(2)} = \begin{bmatrix} 0.090031 \\ -0.334759 \\ 0.244728 \end{bmatrix}$$

The output logits are $\mathbf{z}^{(2)} = W^{(2)T}\mathbf{h} + \mathbf{b}^{(2)}$. Therefore, the gradient with respect to the output weights is

$$\boxed{\nabla_{W^{(2)}} L = \mathbf{h}\boldsymbol{\delta}^{(2)T}}$$

Since $\mathbf{h} = [0.5, 0.5]^T$, the gradient is

$$\nabla_{W^{(2)}} L = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \begin{bmatrix} 0.090031 & -0.334759 & 0.244728 \end{bmatrix}$$

Hence,

$$\boxed{\nabla_{W^{(2)}} L \approx \begin{bmatrix} 0.045015 & -0.167380 & 0.122364 \\ 0.045015 & -0.167380 & 0.122364 \end{bmatrix}}$$

Each entry represents the derivative of the loss with respect to the corresponding output weight. The output-bias gradient is

$$\boxed{\nabla_{\mathbf{b}^{(2)}} L = \boldsymbol{\delta}^{(2)}}$$

The bias gradient is the same as the output-logit gradient because each logit depends on its own bias with derivative 1.

6.2 Propagate the gradient to the hidden layer

The hidden activations affect the loss through the output weights. The chain rule gives

$$\boxed{\nabla_{\mathbf{h}} L = W^{(2)}\boldsymbol{\delta}^{(2)}}$$

Using our weights $W^{(2)} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 3 & 0 \end{bmatrix}$, we calculate the two hidden gradients separately. For hidden unit 1:

$$\frac{\partial L}{\partial h_1} = 0(0.090031) + 1(-0.334759) + 2(0.244728) = \boxed{0.154698}$$

For hidden unit 2:

$$\frac{\partial L}{\partial h_2} = 0(0.090031) + 3(-0.334759) + 0(0.244728) = \boxed{-1.004277}$$

Thus,

$$\nabla_{\mathbf{h}} L \approx \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}$$

The signs differ because the hidden units affect the output classes through different weights.

6.3 Propagate through the ReLU activation

Recall the ReLU function: $h_i = \max(0, z_i^{(1)})$. Its derivative, away from zero, is

$$\mathrm{ReLU}'(z_i^{(1)}) = \begin{cases} 0, & z_i^{(1)} < 0 \\ 1, & z_i^{(1)} > 0 \end{cases}$$

Both pre-activations in our example equal 0.5, so both units are active. The ReLU Jacobian is therefore $J_{\mathrm{ReLU}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. The chain rule gives the gradient of the loss with respect to the hidden pre-activations:

$$\boldsymbol{\delta}^{(1)} = \nabla_{\mathbf{h}} L \odot \mathrm{ReLU}'(\mathbf{z}^{(1)})$$

where $\odot$ denotes element-wise multiplication. Since both derivatives are 1,

$$\boxed{\boldsymbol{\delta}^{(1)} = \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}}$$

If a hidden unit had a negative pre-activation, the corresponding ReLU derivative would be zero, and this gradient would be blocked at that activation for the current input.

6.4 Calculate the gradients for the first layer

The first-layer pre-activation is $\mathbf{z}^{(1)} = W^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}$. Therefore,

$$\boxed{\nabla_{W^{(1)}} L = \mathbf{x}\boldsymbol{\delta}^{(1)T}}$$

Substitute the input and hidden gradients:

$$\nabla_{W^{(1)}} L = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 0.154698 & -1.004277 \end{bmatrix}$$

This produces

$$\boxed{\nabla_{W^{(1)}} L \approx \begin{bmatrix} 0.309396 & -2.008554 \\ 0.154698 & -1.004277 \end{bmatrix}}$$

The first-layer bias gradient is

$$\boxed{\nabla_{\mathbf{b}^{(1)}} L = \boldsymbol{\delta}^{(1)} = \begin{bmatrix} 0.154698 \\ -1.004277 \end{bmatrix}}$$

We now have the gradients for all 15 trainable parameters. This calculation illustrates why backpropagation is efficient: the gradient at each layer reuses the gradient information already computed at the layer above it instead of independently differentiating the entire network for every parameter.

7 Update the parameters using SGD

Backpropagation has calculated the gradients. We now use SGD to update the parameters using the update rule $\theta_{\mathrm{new}} = \theta_{\mathrm{old}} - \eta \nabla_\theta L$. Choose a learning rate of $\eta = 0.1$.

7.1 Update the first-layer weights and biases

$W^{(1)}_{\text{new}} = W^{(1)}_{\text{old}} - 0.1\nabla_{W^{(1)}} L$. Substituting the values,

$$\boxed{W^{(1)}_{\text{new}} \approx \begin{bmatrix} 0.469060 & -0.299145 \\ -0.515470 & 1.100428 \end{bmatrix}}$$

Similarly,

$$\boxed{\mathbf{b}^{(1)}_{\text{new}} \approx \begin{bmatrix} -0.015470 \\ 0.600428 \end{bmatrix}}$$

Notice that a negative gradient produces an increase in the corresponding parameter, because SGD subtracts the gradient.

7.2 Update the output-layer weights and biases

For the second layer, $W^{(2)}_{\text{new}} = W^{(2)}_{\text{old}} - 0.1\nabla_{W^{(2)}} L$. Therefore,

$$\boxed{W^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.004502 & 1.016738 & 1.987764 \\ -0.004502 & 3.016738 & -0.012236 \end{bmatrix}}$$

The output biases become

$$\boxed{\mathbf{b}^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.009003 \\ 0.033476 \\ -0.024473 \end{bmatrix}}$$

All 15 parameters have now been updated.

7.3 Recalculate the prediction

We must perform a new forward pass using these updated parameters. The hidden pre-activations become

$$\mathbf{z}^{(1)}_{\text{new}} \approx \begin{bmatrix} 0.407181 \\ 1.102566 \end{bmatrix}$$

Both remain positive, so

$$\mathbf{h}_{\text{new}} \approx \begin{bmatrix} 0.407181 \\ 1.102566 \end{bmatrix}$$

The output logits are now

$$\mathbf{z}^{(2)}_{\text{new}} \approx \begin{bmatrix} -0.015799 \\ 3.773626 \\ 0.771416 \end{bmatrix}$$

Applying Softmax:

$$\boxed{\mathbf{p}_{\text{new}} \approx \begin{bmatrix} 0.021084 \\ 0.932587 \\ 0.046328 \end{bmatrix}}$$

The probability assigned to DDoS has increased from about 66.52% to 93.26%. The new loss for our DDoS example is

$$\begin{aligned} L_{\text{new}} &= -\ln(0.932587) \\ &\approx \boxed{0.069793} \end{aligned}$$

This is an illustrative single-example update with learning rate 0.1. In practice, SGD trains on many samples over repeated iterations or epochs, and individual updates do not always decrease

State	Cross-entropy loss	Predicted DDoS probability
Before SGD	0.407606	66.52%
After SGD	0.069793	93.26%

the loss. The numerical example connects all the ideas from Sections 6.4–6.5: architecture defines the computation; forward propagation calculates the prediction; the cost function measures the error; the chain rule and Jacobians calculate gradients; backpropagation propagates those gradients efficiently; and SGD updates the parameters.

8 Final picture: Complete network with nodes, edges and calculations

The diagram below shows the initial network used throughout the calculations. The edge labels are the initial weights, the hidden and output nodes display their computed values, and the right-hand panel shows the target and loss. The footer summarizes the backward gradients and the result of one SGD update.

Primary textbook: Goodfellow, Bengio and Courville, Deep Learning, MIT Press, Chapter 6. Read Sections 6.4–6.5.4 for architecture design, approximation properties, Jacobians and backpropagation.